arXiv:1607.06450v1 [stat.ML] 21 Jul 2016

Layer Normalization

Jimmy Lei Ba
University of Toronto
jimmy@psi.toronto.edu

Jamie Ryan Kiros
University of Toronto
rkiros@cs.toronto.edu

Geoffrey E. Hinton
University of Toronto
and Google Inc.
hinton@cs.toronto.edu

Abstract

Training state-of-the-art, deep neural networks is computationally expensive. One way to reduce the training time is to normalize the activities of the neurons. A recently introduced technique called batch normalization uses the distribution of the summed input to a neuron over a mini-batch of training cases to compute a mean and variance which are then used to normalize the summed input to that neuron on each training case. This significantly reduces the training time in feed-forward neural networks. However, the effect of batch normalization is dependent on the mini-batch size and it is not obvious how to apply it to recurrent neural networks. In this paper, we transpose batch normalization into layer normalization by computing the mean and variance used for normalization from all of the summed inputs to the neurons in a layer on a *single* training case. Like batch normalization, we also give each neuron its own adaptive bias and gain which are applied after the normalization but before the non-linearity. Unlike batch normalization, layer normalization performs exactly the same computation at training and test times. It is also straightforward to apply to recurrent neural networks by computing the normalization statistics separately at each time step. Layer normalization is very effective at stabilizing the hidden state dynamics in recurrent networks. Empirically, we show that layer normalization can substantially reduce the training time compared with previously published techniques.

1 Introduction

Deep neural networks trained with some version of Stochastic Gradient Descent have been shown to substantially outperform previous approaches on various supervised learning tasks in computer vision [Krizhevsky et al., 2012] and speech processing [Hinton et al., 2012]. But state-of-the-art deep neural networks often require many days of training. It is possible to speed-up the learning by computing gradients for different subsets of the training cases on different machines or splitting the neural network itself over many machines [Dean et al., 2012], but this can require a lot of communication and complex software. It also tends to lead to rapidly diminishing returns as the degree of parallelization increases. An orthogonal approach is to modify the computations performed in the forward pass of the neural net to make learning easier. Recently, batch normalization [Ioffe and Szegedy, 2015] has been proposed to reduce training time by including additional normalization stages in deep neural networks. The normalization standardizes each summed input using its mean and its standard deviation across the training data. Feedforward neural networks trained using batch normalization converge faster even with simple SGD. In addition to training time improvement, the stochasticity from the batch statistics serves as a regularizer during training.

Despite its simplicity, batch normalization requires running averages of the summed input statistics. In feed-forward networks with fixed depth, it is straightforward to store the statistics separately for each hidden layer. However, the summed inputs to the recurrent neurons in a recurrent neural network (RNN) often vary with the length of the sequence so applying batch normalization to RNNs appears to require different statistics for different time-steps. Furthermore, batch normaliza-

tion cannot be applied to online learning tasks or to extremely large distributed models where the minibatches have to be small.

This paper introduces layer normalization, a simple normalization method to improve the training speed for various neural network models. Unlike batch normalization, the proposed method directly estimates the normalization statistics from the summed inputs to the neurons within a hidden layer so the normalization does not introduce any new dependencies between training cases. We show that layer normalization works well for RNNs and improves both the training time and the generalization performance of several existing RNN models.

2 Background

A feed-forward neural network is a non-linear mapping from a input pattern $\mathbf{x}$ to an output vector y. Consider the l^{th} hidden layer in a deep feed-forward, neural network, and let a^l be the vector representation of the summed inputs to the neurons in that layer. The summed inputs are computed through a linear projection with the weight matrix W^l and the bottom-up inputs h^l given as follows:

$$a_i^l = {w_i^l}^\top h^l \qquad h_i^{l+1} = f(a_i^l + b_i^l) \tag{1}$$

where $f(\cdot)$ is an element-wise non-linear function and w_i^l is the incoming weights to the i^{th} hidden units and b_i^l is the scalar bias parameter. The parameters in the neural network are learnt using gradient-based optimization algorithms with the gradients being computed by back-propagation.

One of the challenges of deep learning is that the gradients with respect to the weights in one layer are highly dependent on the outputs of the neurons in the previous layer especially if these outputs change in a highly correlated way. Batch normalization [Ioffe and Szegedy, 2015] was proposed to reduce such undesirable "covariate shift". The method normalizes the summed inputs to each hidden unit over the training cases. Specifically, for the i^{th} summed input in the l^{th} layer, the batch normalization method rescales the summed inputs according to their variances under the distribution of the data

$$\bar{a}_i^l = \frac{g_i^l}{\sigma_i^l}\left(a_i^l - \mu_i^l\right) \qquad \mu_i^l = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[a_i^l\right] \qquad \sigma_i^l = \sqrt{\mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\left(a_i^l - \mu_i^l\right)^2\right]} \tag{2}$$

where $\bar{a}_i^l$ is normalized summed inputs to the i^{th} hidden unit in the l^{th} layer and g_i is a gain parameter scaling the normalized activation before the non-linear activation function. Note the expectation is under the whole training data distribution. It is typically impractical to compute the expectations in Eq. (2) exactly, since it would require forward passes through the whole training dataset with the current set of weights. Instead, μ and σ are estimated using the empirical samples from the current mini-batch. This puts constraints on the size of a mini-batch and it is hard to apply to recurrent neural networks.

3 Layer normalization

We now consider the layer normalization method which is designed to overcome the drawbacks of batch normalization.

Notice that changes in the output of one layer will tend to cause highly correlated changes in the summed inputs to the next layer, especially with ReLU units whose outputs can change by a lot. This suggests the "covariate shift" problem can be reduced by fixing the mean and the variance of the summed inputs within each layer. We, thus, compute the layer normalization statistics over all the hidden units in the same layer as follows:

$$\mu^l = \frac{1}{H}\sum_{i=1}^{H} a_i^l \qquad \sigma^l = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^l - \mu^l\right)^2} \tag{3}$$

where H denotes the number of hidden units in a layer. The difference between Eq. (2) and Eq. (3) is that under layer normalization, all the hidden units in a layer share the same normalization terms μ and σ, but different training cases have different normalization terms. Unlike batch normalization, layer normaliztion does not impose any constraint on the size of a mini-batch and it can be used in the pure online regime with batch size 1.

3.1 Layer normalized recurrent neural networks

The recent sequence to sequence models [Sutskever et al., 2014] utilize compact recurrent neural networks to solve sequential prediction problems in natural language processing. It is common among the NLP tasks to have different sentence lengths for different training cases. This is easy to deal with in an RNN because the same weights are used at every time-step. But when we apply batch normalization to an RNN in the obvious way, we need to to compute and store separate statistics for each time step in a sequence. This is problematic if a test sequence is longer than any of the training sequences. Layer normalization does not have such problem because its normalization terms depend only on the summed inputs to a layer at the current time-step. It also has only one set of gain and bias parameters shared over all time-steps.

In a standard RNN, the summed inputs in the recurrent layer are computed from the current input $\mathbf{x}^t$ and previous vector of hidden states $\mathbf{h}^{t-1}$ which are computed as $\mathbf{a}^t = W_{hh}h^{t-1} + W_{xh}\mathbf{x}^t$. The layer normalized recurrent layer re-centers and re-scales its activations using the extra normalization terms similar to Eq. (3):

$$\mathbf{h}^t = f\left[\frac{\mathbf{g}}{\sigma^t} \odot \left(\mathbf{a}^t - \mu^t\right) + \mathbf{b}\right] \qquad \mu^t = \frac{1}{H}\sum_{i=1}^{H} a_i^t \qquad \sigma^t = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^t - \mu^t\right)^2} \tag{4}$$

where W_{hh} is the recurrent hidden to hidden weights and W_{xh} are the bottom up input to hidden weights. $\odot$ is the element-wise multiplication between two vectors. $\mathbf{b}$ and $\mathbf{g}$ are defined as the bias and gain parameters of the same dimension as $\mathbf{h}^t$.

In a standard RNN, there is a tendency for the average magnitude of the summed inputs to the recurrent units to either grow or shrink at every time-step, leading to exploding or vanishing gradients. In a layer normalized RNN, the normalization terms make it invariant to re-scaling all of the summed inputs to a layer, which results in much more stable hidden-to-hidden dynamics.

4 Related work

Batch normalization has been previously extended to recurrent neural networks [Laurent et al., 2015, Amodei et al., 2015, Cooijmans et al., 2016]. The previous work [Cooijmans et al., 2016] suggests the best performance of recurrent batch normalization is obtained by keeping independent normalization statistics for each time-step. The authors show that initializing the gain parameter in the recurrent batch normalization layer to 0.1 makes significant difference in the final performance of the model. Our work is also related to weight normalization [Salimans and Kingma, 2016]. In weight normalization, instead of the variance, the L2 norm of the incoming weights is used to normalize the summed inputs to a neuron. Applying either weight normalization or batch normalization using expected statistics is equivalent to have a different parameterization of the original feed-forward neural network. Re-parameterization in the ReLU network was studied in the Path-normalized SGD [Neyshabur et al., 2015]. Our proposed layer normalization method, however, is not a re-parameterization of the original neural network. The layer normalized model, thus, has different invariance properties than the other methods, that we will study in the following section.

5 Analysis

In this section, we investigate the invariance properties of different normalization schemes.

5.1 Invariance under weights and data transformations

The proposed layer normalization is related to batch normalization and weight normalization. Although, their normalization scalars are computed differently, these methods can be summarized as normalizing the summed inputs a_i to a neuron through the two scalars μ and σ. They also learn an adaptive bias b and gain g for each neuron after the normalization.

$$h_i = f(\frac{g_i}{\sigma_i}\left(a_i - \mu_i\right) + b_i) \tag{5}$$

Note that for layer normalization and batch normalization, μ and σ is computed according to Eq. 2 and 3. In weight normalization, μ is 0, and $\sigma = \|w\|_2$.

	Weight matrix re-scaling	Weight matrix re-centering	Weight vector re-scaling	Dataset re-scaling	Dataset re-centering	Single training case re-scaling
Batch norm	Invariant	No	Invariant	Invariant	Invariant	No
Weight norm	Invariant	No	Invariant	No	No	No
Layer norm	Invariant	Invariant	No	Invariant	No	Invariant

Table 1: Invariance properties under the normalization methods.

Table 1 highlights the following invariance results for three normalization methods.

Weight re-scaling and re-centering: First, observe that under batch normalization and weight normalization, any re-scaling to the incoming weights w_i of a single neuron has no effect on the normalized summed inputs to a neuron. To be precise, under batch and weight normalization, if the weight vector is scaled by δ, the two scalar μ and σ will also be scaled by δ. The normalized summed inputs stays the same before and after scaling. So the batch and weight normalization are invariant to the re-scaling of the weights. Layer normalization, on the other hand, is not invariant to the individual scaling of the single weight vectors. Instead, layer normalization is invariant to scaling of the entire weight matrix and invariant to a shift to all of the incoming weights in the weight matrix. Let there be two sets of model parameters θ, θ' whose weight matrices W and W' differ by a scaling factor δ and all of the incoming weights in W' are also shifted by a constant vector $\boldsymbol{\gamma}$, that is $W' = \delta W + \mathbf{1}\boldsymbol{\gamma}^\top$. Under layer normalization, the two models effectively compute the same output:

$$\begin{aligned}\mathbf{h}' =& f(\frac{\mathbf{g}}{\sigma'}\left(W'\mathbf{x} - \mu'\right) + \mathbf{b}) = f(\frac{\mathbf{g}}{\sigma'}\left((\delta W + \mathbf{1}\boldsymbol{\gamma}^\top)\mathbf{x} - \mu'\right) + \mathbf{b}) \\ =& f(\frac{\mathbf{g}}{\sigma}\left(W\mathbf{x} - \mu\right) + \mathbf{b}) = \mathbf{h}. \end{aligned} \tag{6}$$

Notice that if normalization is only applied to the input before the weights, the model will not be invariant to re-scaling and re-centering of the weights.

Data re-scaling and re-centering: We can show that all the normalization methods are invariant to re-scaling the dataset by verifying that the summed inputs of neurons stays constant under the changes. Furthermore, layer normalization is invariant to re-scaling of individual training cases, because the normalization scalars μ and σ in Eq. (3) only depend on the current input data. Let $\mathbf{x}'$ be a new data point obtained by re-scaling $\mathbf{x}$ by δ. Then we have,

$$h_i' = f(\frac{g_i}{\sigma'}\left(w_i^\top \mathbf{x}' - \mu'\right) + b_i) = f(\frac{g_i}{\delta\sigma}\left(\delta w_i^\top \mathbf{x} - \delta\mu\right) + b_i) = h_i. \tag{7}$$

It is easy to see re-scaling individual data points does not change the model's prediction under layer normalization. Similar to the re-centering of the weight matrix in layer normalization, we can also show that batch normalization is invariant to re-centering of the dataset.

5.2 Geometry of parameter space during learning

We have investigated the invariance of the model's prediction under re-centering and re-scaling of the parameters. Learning, however, can behave very differently under different parameterizations, even though the models express the same underlying function. In this section, we analyze learning behavior through the geometry and the manifold of the parameter space. We show that the normalization scalar σ can implicitly reduce learning rate and makes learning more stable.

5.2.1 Riemannian metric

The learnable parameters in a statistical model form a smooth manifold that consists of all possible input-output relations of the model. For models whose output is a probability distribution, a natural way to measure the separation of two points on this manifold is the Kullback-Leibler divergence between their model output distributions. Under the KL divergence metric, the parameter space is a Riemannian manifold.

The curvature of a Riemannian manifold is entirely captured by its Riemannian metric, whose quadratic form is denoted as ds^2. That is the infinitesimal distance in the tangent space at a point in the parameter space. Intuitively, it measures the changes in the model output from the parameter space along a tangent direction. The Riemannian metric under KL was previously studied [Amari, 1998] and was shown to be well approximated under second order Taylor expansion using the Fisher

information matrix:

$$ds^2 = \mathrm{D}_{\mathrm{KL}}\big[P(y \,|\, \mathbf{x};\ \theta) \| P(y \,|\, \mathbf{x};\ \theta + \delta)\big] \approx \frac{1}{2}\delta^\top F(\theta)\delta, \tag{8}$$

$$F(\theta) = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x}), y\sim P(y\,|\,\mathbf{x})}\left[\frac{\partial \log P(y \,|\, \mathbf{x};\ \theta)}{\partial \theta}\frac{\partial \log P(y \,|\, \mathbf{x};\ \theta)}{\partial \theta}^\top\right], \tag{9}$$

where, δ is a small change to the parameters. The Riemannian metric above presents a geometric view of parameter spaces. The following analysis of the Riemannian metric provides some insight into how normalization methods could help in training neural networks.

5.2.2 The geometry of normalized generalized linear models

We focus our geometric analysis on the generalized linear model. The results from the following analysis can be easily applied to understand deep neural networks with block-diagonal approximation to the Fisher information matrix, where each block corresponds to the parameters for a single neuron.

A generalized linear model (GLM) can be regarded as parameterizing an output distribution from the exponential family using a weight vector w and bias scalar b. To be consistent with the previous sections, the log likelihood of the GLM can be written using the summed inputs a as the following:

$$\log P(y \,|\, \mathbf{x};\ w, b) = \frac{(a+b)y - \eta(a+b)}{\phi} + c(y, \phi), \tag{10}$$

$$\mathbb{E}[y \,|\, \mathbf{x}] = f(a+b) = f(w^\top \mathbf{x} + b),\ \ \mathrm{Var}[y \,|\, \mathbf{x}] = \phi f'(a+b), \tag{11}$$

where, $f(\cdot)$ is the transfer function that is the analog of the non-linearity in neural networks, $f'(\cdot)$ is the derivative of the transfer function, $\eta(\cdot)$ is a real valued function and $c(\cdot)$ is the log partition function. ϕ is a constant that scales the output variance. Assume a H-dimensional output vector $\mathbf{y} = [y_1, y_2, \cdots, y_H]$ is modeled using H independent GLMs and $\log P(\mathbf{y} \,|\, \mathbf{x};\ W, \mathbf{b}) = \sum_{i=1}^{H} \log P(y_i \,|\, \mathbf{x};\ w_i, b_i)$. Let W be the weight matrix whose rows are the weight vectors of the individual GLMs, $\mathbf{b}$ denote the bias vector of length H and $\mathrm{vec}(\cdot)$ denote the Kronecker vector operator. The Fisher information matrix for the multi-dimensional GLM with respect to its parameters $\theta = [w_1^\top, b_1, \cdots, w_H^\top, b_H]^\top = \mathrm{vec}([W, \mathbf{b}]^\top)$ is simply the expected Kronecker product of the data features and the output covariance matrix:

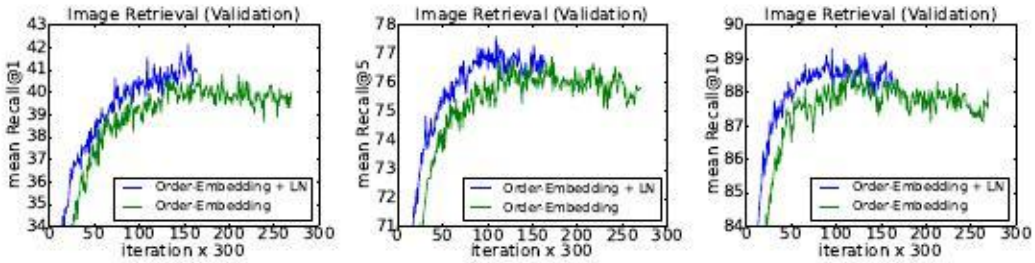
$$F(\theta) = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\frac{\mathrm{Cov}[\mathbf{y} \,|\, \mathbf{x}]}{\phi^2} \otimes \begin{bmatrix}\mathbf{x}\mathbf{x}^\top & \mathbf{x} \\ \mathbf{x}^\top & 1\end{bmatrix}\right]. \tag{12}$$

We obtain normalized GLMs by applying the normalization methods to the summed inputs a in the original model through μ and σ. Without loss of generality, we denote $\bar{F}$ as the Fisher information matrix under the normalized multi-dimensional GLM with the additional gain parameters $\theta = \mathrm{vec}([W, \mathbf{b}, \mathbf{g}]^\top)$:

$$\bar{F}(\theta) = \begin{bmatrix}\bar{F}_{11} & \cdots & \bar{F}_{1H} \\ \vdots & \ddots & \vdots \\ \bar{F}_{H1} & \cdots & \bar{F}_{HH}\end{bmatrix}, \quad \bar{F}_{ij} = \mathop{\mathbb{E}}_{\mathbf{x}\sim P(\mathbf{x})}\left[\frac{\mathrm{Cov}[y_i,\ y_j \ |\, \mathbf{x}]}{\phi^2}\begin{bmatrix}\frac{g_i g_j}{\sigma_i\sigma_j}\chi_i\chi_j^\top & \chi_i\frac{g_i}{\sigma_i} & \chi_i\frac{g_i(a_j-\mu_j)}{\sigma_i\sigma_j} \\ \chi_j^\top\frac{g_j}{\sigma_j} & 1 & \frac{a_j-\mu_j}{\sigma_j} \\ \chi_j^\top\frac{g_j(a_i-\mu_i)}{\sigma_i\sigma_j} & \frac{a_i-\mu_i}{\sigma_i} & \frac{(a_i-\mu_i)(a_j-\mu_j)}{\sigma_i\sigma_j}\end{bmatrix}\right] \tag{13}$$

$$\chi_i = \mathbf{x} - \frac{\partial \mu_i}{\partial w_i} - \frac{a_i - \mu_i}{\sigma_i}\frac{\partial \sigma_i}{\partial w_i}. \tag{14}$$

Implicit learning rate reduction through the growth of the weight vector: Notice that, comparing to standard GLM, the block $\bar{F}_{ij}$ along the weight vector w_i direction is scaled by the gain parameters and the normalization scalar σ_i. If the norm of the weight vector w_i grows twice as large, even though the model's output remains the same, the Fisher information matrix will be different. The curvature along the w_i direction will change by a factor of $\frac{1}{2}$ because the σ_i will also be twice as large. As a result, for the same parameter update in the normalized model, the norm of the weight vector effectively controls the learning rate for the weight vector. During learning, it is harder to change the orientation of the weight vector with large norm. The normalization methods, therefore,



(a) Recall@1 (b) Recall@5 (c) Recall@10

Figure 1: Recall@K curves using order-embeddings with and without layer normalization.

	MSCOCO							
	Caption Retrieval				Image Retrieval			
Model	**R@1**	**R@5**	**R@10**	**Mean r**	**R@1**	**R@5**	**R@10**	**Mean r**
Sym [Vendrov et al., 2016]	45.4		88.7	5.8	36.3		85.8	9.0
OE [Vendrov et al., 2016]	46.7		88.9	5.7	37.9		85.9	8.1
OE (ours)	46.6	79.3	89.1	5.2	37.8	73.6	85.7	7.9
OE + LN	**48.5**	**80.6**	**89.8**	**5.1**	**38.9**	**74.3**	**86.3**	**7.6**

Table 2: Average results across 5 test splits for caption and image retrieval. **R@K** is Recall@K (high is good). **Mean** r is the mean rank (low is good). Sym corresponds to the symmetric baseline while OE indicates order-embeddings.

have an implicit "early stopping" effect on the weight vectors and help to stabilize learning towards convergence.

Learning the magnitude of incoming weights: In normalized models, the magnitude of the incoming weights is explicitly parameterized by the gain parameters. We compare how the model output changes between updating the gain parameters in the normalized GLM and updating the magnitude of the equivalent weights under original parameterization during learning. The direction along the gain parameters in F captures the geometry for the magnitude of the incoming weights. We show that Riemannian metric along the magnitude of the incoming weights for the standard GLM is scaled by the norm of its input, whereas learning the gain parameters for the batch normalized and layer normalized models depends only on the magnitude of the prediction error. Learning the magnitude of incoming weights in the normalized model is therefore, more robust to the scaling of the input and its parameters than in the standard model. See Appendix for detailed derivations.

6 Experimental results

We perform experiments with layer normalization on 6 tasks, with a focus on recurrent neural networks: image-sentence ranking, question-answering, contextual language modelling, generative modelling, handwriting sequence generation and MNIST classification. Unless otherwise noted, the default initialization of layer normalization is to set the adaptive gains to 1 and the biases to 0 in the experiments.

6.1 Order embeddings of images and language

In this experiment, we apply layer normalization to the recently proposed order-embeddings model of Vendrov et al. [2016] for learning a joint embedding space of images and sentences. We follow the same experimental protocol as Vendrov et al. [2016] and modify their publicly available code to incorporate layer normalization [1] which utilizes Theano [Team et al., 2016]. Images and sentences from the Microsoft COCO dataset [Lin et al., 2014] are embedded into a common vector space, where a GRU [Cho et al., 2014] is used to encode sentences and the outputs of a pre-trained VGG ConvNet [Simonyan and Zisserman, 2015] (10-crop) are used to encode images. The order-embedding model represents images and sentences as a 2-level partial ordering and replaces the cosine similarity scoring function used in Kiros et al. [2014] with an asymmetric one.

[1] https://github.com/ivendrov/order-embedding